

6 PORTFOLIO PROJECTS EVERY EE STUDENT NEEDS

Build these. Get hired.

Full breakdowns, components, and step-by-step guides

By @1nska

NYU Shanghai — 2026

Why Your Portfolio Matters More Than Your GPA

Here's the truth that most EE programs won't tell you: the students who land the best jobs, internships, and research positions aren't the ones with the highest grades. They're the ones who can point to something they built and say "I made this, and here's how it works."

A strong portfolio does three things: it proves you can take a project from concept to completion, it shows you understand real-world constraints (not just textbook problems), and it gives interviewers something concrete to talk about instead of generic behavioral questions.

These 6 projects are designed to cover the key areas employers look for in EE graduates: embedded systems, signal processing, PCB design, power electronics, IoT, and control systems. Build even 2-3 of these and you'll be ahead of 90% of applicants.

How to use this guide: Each project includes what you need, the skills it proves, step-by-step build instructions, and a bonus tip to take it further. Start with whichever excites you most — momentum beats perfection.

#1

Smart Energy Monitor

IOT • SENSORS • DATA VISUALIZATION

Why recruiters love this: It shows you can integrate hardware sensors with software dashboards — the exact skill set that IoT companies and smart grid startups are desperate for.

Build a device that clips onto your home's electrical panel (or a single appliance) and tracks real-time power consumption. The data gets sent wirelessly to a web dashboard where you can see usage patterns, peak hours, and estimated costs.

What you need:

- ESP32 or Arduino with WiFi module
- ACS712 current sensor (or non-invasive CT clamp sensor)
- Voltage sensor module (ZMPT101B)
- OLED display (optional, for local readout)
- Breadboard, jumper wires, USB power supply
- Free cloud platform (ThingSpeak, Firebase, or Blynk)

Skills it proves:

- Sensor integration and calibration
- Wireless communication protocols (WiFi, MQTT)
- Real-time data processing on embedded systems
- Full-stack thinking — hardware to cloud to dashboard
- Power measurement fundamentals (real vs apparent power)

How to build it:

Step 1: Start with the current sensor. Wire the ACS712 to your ESP32, read analog values, and calibrate against a known load (like a 60W bulb). Get accurate current readings first.

Step 2: Add voltage sensing. Wire the ZMPT101B and combine with current data to calculate real power ($P = V \times I \times \text{power factor}$). Log readings every second.

Step 3: Set up wireless transmission. Connect your ESP32 to WiFi and send data to ThingSpeak or Firebase using MQTT or HTTP POST requests. Verify data arrives in the cloud.

Step 4: Build the dashboard. Use ThingSpeak's built-in charts or build a simple web dashboard with HTML/JavaScript that pulls from your cloud database. Show real-time watts, daily kWh, and estimated cost.

Step 5: Add alerting. Set up notifications (email or push) when power consumption exceeds a threshold. This shows you understand practical monitoring systems.

Step 6: Package it. Put the electronics in a small enclosure. Document the full build with photos, circuit diagrams, and code on GitHub.

Bonus: Add ML-based anomaly detection that flags unusual consumption patterns — like a failing appliance drawing more power than normal. This bridges into the ML space and makes the project 10x more impressive.

#2

Gesture-Controlled Robot Arm

SIGNAL PROCESSING • EMBEDDED SYSTEMS • HARDWARE

Why recruiters love this: It combines signal processing, embedded programming, and mechanical design in one project. It's visually impressive in demos and shows deep understanding of sensor-to-actuator pipelines.

Build a robotic arm that you control with your forearm muscles. EMG (electromyography) sensors on your arm detect muscle contractions, an Arduino processes the signals in real time, and servo motors move the robotic arm to mirror your gestures.

What you need:

- MyoWare 2.0 EMG sensor (or DIY EMG circuit with instrumentation amp)
- Arduino Mega or ESP32
- 4-6 servo motors (SG90 or MG996R for more torque)
- 3D-printed or laser-cut robot arm frame (files available free online)
- Servo driver board (PCA9685) for controlling multiple servos
- 9V battery pack or bench power supply

Skills it proves:

- Analog signal processing — filtering and amplifying biological signals
- Real-time embedded programming with tight timing constraints
- Servo motor control and PWM fundamentals
- Human-machine interface design
- System integration across multiple subsystems

How to build it:

Step 1: Start with the EMG sensor. Connect the MyoWare to your Arduino and read raw muscle signals. Plot them on the serial monitor to understand the signal patterns when you flex, extend, and relax.

Step 2: Filter the signal. Implement a moving average filter or bandpass filter in code to clean up the noisy EMG data. You need clean, reliable threshold detection.

Step 3: Map signals to movements. Define gesture profiles — e.g., light flex = open hand, hard flex = close grip, wrist rotation = arm rotate. Create a calibration routine that adapts to different users.

Step 4: Build or assemble the arm. Use a free 3D-printable design (search Thingiverse for 'robot arm Arduino'). Attach servos to each joint and wire them through the PCA9685 driver board.

Step 5: Connect everything. Write the main control loop: read EMG → filter → classify gesture → move corresponding servo. Focus on smooth, responsive movement.

Step 6: Demo and document. Record a video of you controlling the arm with your muscles. This is your portfolio showpiece. Upload code, wiring diagrams, and the demo video to GitHub.

Bonus: Add a second EMG sensor on the other arm for two-handed control, or implement a simple ML classifier (using TensorFlow Lite on the ESP32) that recognizes more complex gestures with higher accuracy.

#3

Custom PCB Synthesizer

PCB DESIGN • ANALOG CIRCUITS • AUDIO

Why recruiters love this: PCB design is the #1 practical skill EE employers test for. Designing a board that actually makes sound proves you can go from schematic to layout to physical product — the full hardware development cycle.

Design and fabricate a custom PCB that generates audio waveforms (sine, square, triangle, sawtooth). Add potentiometers for frequency and volume control, and an audio jack output so you can plug it into a speaker or headphones. It's a functioning musical instrument you designed from scratch.

What you need:

- KiCad (free PCB design software)
- Op-amps (TL072 or LM358)
- 555 timer IC and/or XR2206 function generator IC
- Potentiometers (10K and 100K)
- 3.5mm audio jack, capacitors, resistors
- PCB fabrication service (JLCPCB or PCBWay — under \$5 for 5 boards)

Skills it proves:

- Schematic design and circuit analysis
- PCB layout — trace routing, ground planes, component placement
- Analog circuit design — oscillators, filters, amplifiers
- Manufacturing awareness — DFM, tolerances, BOM management
- Debugging physical hardware with oscilloscope and multimeter

How to build it:

Step 1: Design the oscillator circuit. Start in KiCad with a 555 timer or XR2206-based oscillator. Simulate in LTSpice to verify your waveform outputs before committing to a PCB.

Step 2: Add control circuits. Design potentiometer-based frequency control (variable resistor in the timing circuit) and a volume control stage using an op-amp attenuator.

Step 3: Add waveform selection. Use a rotary switch or DIP switches to select between waveform types. Each position routes a different shaping circuit into the output.

Step 4: Layout the PCB. Route your schematic in KiCad's PCB editor. Keep analog and digital sections separated, use proper ground planes, and keep trace widths appropriate for audio signals.

Step 5: Order and assemble. Send gerber files to JLCPCB. While waiting, order components. When boards arrive, hand-solder components and test each section incrementally.

Step 6: Test and tune. Use an oscilloscope to verify waveform shape, frequency range, and amplitude. Debug any distortion or noise issues. Document your design decisions and test results.

Bonus: Add MIDI input so the synthesizer can be played from a keyboard or DAW. This transforms it from a demo project into a genuinely usable instrument and shows digital communication protocol knowledge.

#4

MPPT Solar Charge Controller

POWER ELECTRONICS • RENEWABLE ENERGY • CONTROL SYSTEMS

Why recruiters love this: Power electronics is one of the highest-demand EE specializations. Building an MPPT controller shows you understand DC-DC conversion, control loops, and renewable energy systems — all in one project.

Build a Maximum Power Point Tracking (MPPT) solar charge controller that optimizes energy harvest from a solar panel and efficiently charges a battery. Unlike simple PWM controllers, MPPT dynamically adjusts the operating point to extract maximum power regardless of conditions.

What you need:

- Small solar panel (10-20W)
- Arduino or STM32 microcontroller
- Buck or buck-boost converter components (MOSFET, inductor, diode, capacitors)
- INA219 current/voltage sensor module
- 12V lead-acid or lithium battery
- LCD display for status readout

Skills it proves:

- DC-DC converter design and analysis
- Control system implementation (Perturb & Observe algorithm)
- Power MOSFET driving and gate driver circuits
- Efficiency measurement and optimization
- Real-world renewable energy system understanding

How to build it:

Step 1: Build the buck converter. Design and assemble a basic DC-DC buck converter on a breadboard. Use a MOSFET, Schottky diode, and inductor. Verify voltage step-down with a bench power supply before connecting the solar panel.

Step 2: Add sensing. Wire INA219 modules to measure both the solar panel voltage/current and the battery voltage/current. Log these values to verify your measurements are accurate.

Step 3: Implement MPPT algorithm. Code the Perturb & Observe algorithm — slightly adjust the duty cycle, measure if power increased or decreased, and continue in the direction that increases power. This is the core of the project.

Step 4: Add battery management. Implement charging stages — bulk, absorption, float. Add overvoltage and overcurrent protection in software. Add a physical fuse as a safety backup.

Step 5: Build the display interface. Show solar power in, battery voltage, charging current, and efficiency on an LCD. Log data to an SD card for performance analysis over time.

Step 6: Test under real conditions. Take it outside, connect to a real solar panel, and charge a real battery. Measure and document your MPPT efficiency vs. a simple direct-connect setup.

Bonus: Add WiFi connectivity and log performance data to a cloud dashboard. Compare your MPPT efficiency across different weather conditions and times of day. This data-driven approach impresses power systems employers.

#5

Software-Defined Radio Receiver

RF • SIGNAL PROCESSING • COMMUNICATIONS

Why recruiters love this: RF and communications engineering is a specialized, high-paying niche. Building an SDR receiver proves you understand the full signal chain — from antenna to demodulated audio — and can work with real-world RF signals.

Build a software-defined radio that can receive and demodulate FM radio, aircraft communications (ADS-B), weather satellite images, and more. You'll process raw RF data in software, implementing filters, demodulators, and decoders that traditionally require expensive dedicated hardware.

What you need:

- RTL-SDR dongle (RTL2832U-based, ~\$25)
- Antenna (dipole for FM, or build a QFH antenna for weather satellites)
- Python with NumPy, SciPy, and matplotlib
- GNU Radio (free, open-source SDR framework)
- Computer with USB port
- Optional: LNA (low noise amplifier) for weak signals

Skills it proves:

- RF fundamentals — frequency, bandwidth, modulation schemes
- Digital signal processing — FFT, filtering, sampling theory
- Communication protocols and demodulation techniques
- Python for real-time signal processing
- Antenna design basics

How to build it:

Step 1: Set up the RTL-SDR. Install drivers and GNU Radio. Verify you can receive FM radio — tune to a local station and confirm you hear audio. This validates your entire receive chain.

Step 2: Build an FM demodulator from scratch. Using Python and NumPy, read raw IQ samples from the RTL-SDR and implement FM demodulation manually — without GNU Radio blocks. This proves you understand the DSP, not just the tools.

Step 3: Visualize the spectrum. Build a real-time spectrum analyzer using FFT. Display a waterfall plot showing signal activity across a frequency range. This is a powerful demo piece.

Step 4: Decode ADS-B aircraft signals. Tune to 1090 MHz and decode aircraft position and identification data. Parse the binary protocol and plot aircraft positions on a map.

Step 5: Receive a weather satellite image. Build or buy a QFH antenna, receive a NOAA satellite pass at 137 MHz, and decode the APT signal into an actual weather image. This is the showstopper demo.

Step 6: Document everything. Create a GitHub repo with your Python code, GNU Radio flowgraphs, antenna design, and example outputs (spectrum plots, decoded images, aircraft tracks).

Bonus: Implement your own digital modulation scheme (BPSK or QPSK) and transmit/receive data between two SDRs. Note: only transmit at legal power levels or use a direct cable connection to stay compliant with regulations.

#6

PID-Controlled Balancing Robot

CONTROL SYSTEMS • ROBOTICS • EMBEDDED

Why recruiters love this: Control systems are fundamental to EE and notoriously hard to implement well in practice. A self-balancing robot is the perfect proof that you can tune a real control loop — not just solve transfer functions on paper.

Build a two-wheeled robot that balances itself upright using a PID control loop. An IMU (accelerometer + gyroscope) measures tilt angle, a microcontroller runs the PID algorithm, and DC motors with encoders make corrections in real time. It's the inverted pendulum problem — brought to life.

What you need:

- Arduino Mega or ESP32
- MPU6050 IMU (accelerometer + gyroscope)
- 2x DC gear motors with encoders
- L298N or TB6612FNG motor driver
- Robot chassis (3D print or use a kit frame)
- LiPo battery (7.4V 2S)
- Wheels with rubber tires for grip

Skills it proves:

- PID control theory and real-world tuning
- Sensor fusion (complementary or Kalman filter)
- Real-time embedded control with tight loop timing
- Motor control and H-bridge driver circuits
- System dynamics and stability analysis

How to build it:

Step 1: Get IMU data. Wire the MPU6050 to your Arduino, read raw accelerometer and gyroscope values. Implement a complementary filter to fuse both sensors into a stable tilt angle estimate.

Step 2: Build the chassis. Mount the motors, wheels, battery, and electronics. Keep the center of gravity high — this actually makes balancing easier (longer pendulum = slower fall). Ensure the IMU is mounted at the pivot point.

Step 3: Implement basic PID. Start with proportional control only (P). Set I and D to zero. Gradually increase P until the robot starts reacting to tilts. It will oscillate — that's expected.

Step 4: Tune the full PID loop. Add derivative (D) to dampen oscillations. Add integral (I) to eliminate steady-state lean. Use the Ziegler-Nichols method or manual tuning. This is where you really learn control systems.

Step 5: Add encoder feedback. Use motor encoders to implement a position hold — so the robot doesn't just balance but stays in one place. This is a cascaded control loop (inner: balance, outer: position).

Step 6: Demonstrate and document. Film the robot balancing, recovering from pushes, and holding position. Show your PID tuning process and control system block diagram on GitHub. Include plots of angle vs. time during tuning.

Bonus: Add Bluetooth or WiFi remote control so you can drive the robot while it self-balances. Or implement LQR (Linear Quadratic Regulator) control instead of PID and compare performance — this shows advanced control theory knowledge.

What To Do Next

Don't try to build all 6 at once. Pick the one that excites you most, order the parts today, and start this weekend. A finished project beats a perfect plan every time.

When you finish a project, document it properly: clean code on GitHub, a detailed README with photos and diagrams, and a short demo video. That package is what gets you interviews.

If you want more guides like this — on building startups, career strategy for engineers, or bridging business and engineering — follow me on Instagram. I'm documenting the entire journey.

Quick Reference

#	Project	Key Skill Area	Difficulty	Cost
1	Smart Energy Monitor	IoT + Data	Beginner	\$20-30
2	Gesture Robot Arm	Signal Processing	Intermediate	\$50-80
3	PCB Synthesizer	PCB Design	Intermediate	\$15-25
4	MPPT Solar Controller	Power Electronics	Advanced	\$40-60
5	SDR Receiver	RF + Comms	Intermediate	\$30-40
6	Balancing Robot	Control Systems	Advanced	\$50-70

Follow @1nska on Instagram for more project guides, build logs, and real talk about building as a student. DM me "BUILD" and I'll help you pick the right project for your experience level.

— Enes (@1nska)
NYU Shanghai, 2026